



Name: Fatima Abubakar

Registration Number: 01026077594DE

Faculty: Science and Computing

Department: Computer Science (Network Technology and Cybersecurity)

Course: Computer Organization (Linux Server/OS)

Project Proposal

This practical project focuses on implementing automated data backup, security hardening, and storage lifecycle management on a Debian Linux server environment. The project uses Bash shell scripting, POSIX access control lists, compressed tape archiving (tar), and automated retention rules (find) to protect user directories against accidental data loss, enforce least-privilege permissions, and ensure continuous log auditing for system reliability.

Introduction

As part of the Practical Skill Application (PSA) curriculum, I executed an administrative engineering task aimed at addressing critical system recovery and data integrity risks within a multi-user Linux system. Performing manual system backups exposes enterprise environments to human error, missed schedules, and inconsistent access protections.

This PSA project implemented an automated shell script that isolates root-level storage, compresses active filesystem trees, enforces strict cryptographic file modes (700 and 600), logs execution statuses, and purges outdated archives to maintain predictable disk utilization.

Problem Statement

During system observation, the host environment exhibited several operational risks common to unmanaged Linux nodes:

- **Absence of Centralized Archiving:** User directories (/home) lacked scheduled, self-contained snapshots, exposing user files to accidental deletion or filesystem corruption.
- **Excessive Access Exposure:** Default backup locations risked world-readable exposures, permitting unauthorized non-root accounts to inspect or extract sensitive archives.
- **Storage Exhaustion Hazards:** Storing unmonitored backups without retention policies leads to storage exhaustion and service outages.
- **Lack of Audit Observability:** Failed or incomplete compression operations went unrecorded, making audit verification and disaster recovery validation impossible.

Solution

To remediate these operational vulnerabilities, an automated script (backup.sh) was engineered and validated through the root shell environment:

1. Restricted Storage Provisioning:

- Created a dedicated system backup vault at /var/backups/system_archive.
- Enforced directory mode 700 (drwx-----) owned by root:root, restricting traversal and read permissions entirely to administrative processes.

2. Automated Compression & Strict File Modes:

- Automated snapshot packaging of /home using tar -czf with microsecond-level timestamps (%Y%m%d_%H%M%S) to prevent archive collisions.
- Enforced file mode 600 (-rw-----) on every generated archive file to protect data at rest.

3. Execution Safety and Error Handling:

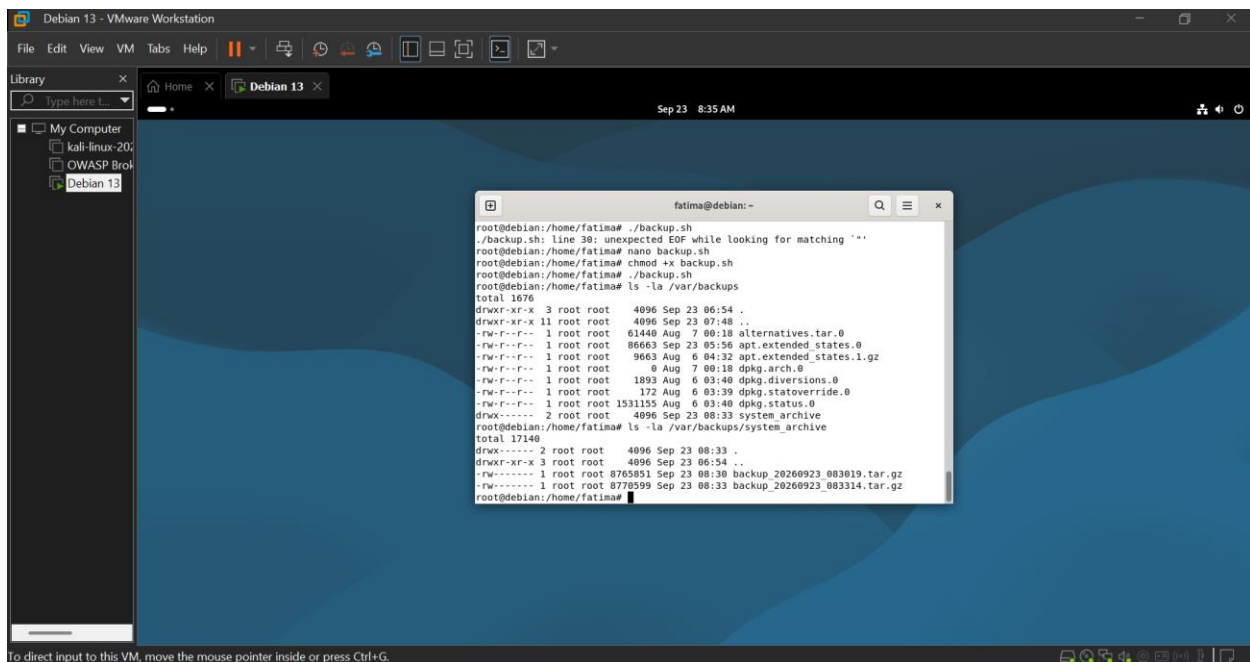
- Integrated set -euo pipefail to halt script execution instantly upon unbound variable expansion or command pipeline failure.
- Built status-verification branches (if [\$? -eq 0]) to validate process exit codes.

4. Retention Policies & Audit Logging:

- Configured find "\$BACKUP_DIR" -type f -name "*.tar.gz" -mtime +30 -exec rm -f {} \; to automatically rotate and delete backups older than 30 days.
- Routed standard system records directly to /var/backup.log to track successful backup sizes, execution timestamps, and cleanup operations.

Practical Verification & Output Evidence

Testing on Debian GNU/Linux confirmed successful script execution, correct directory permissions, and real archive creation:



```

fatima@debian:~$ ./backup.sh
./backup.sh: line 30: unexpected EOF while looking for matching `''
root@debian:/home/fatima# nano backup.sh
root@debian:/home/fatima# chmod +x backup.sh
root@debian:/home/fatima# ./backup.sh
root@debian:/home/fatima# ls -la /var/backups
total 1676
drwxr-xr-x 3 root root 4096 Sep 23 06:54 .
drwxr-xr-x 11 root root 4096 Sep 23 07:48 ..
-rw-r--r-- 1 root root 61440 Aug 7 00:18 alternatives.tar.0
-rw-r--r-- 1 root root 86663 Sep 23 05:56 apt.extended_states.0
-rw-r--r-- 1 root root 9663 Aug 6 04:32 apt.extended_states.l.gz
-rw-r--r-- 1 root root 0 Aug 7 00:18 dpkg.arch.0
-rw-r--r-- 1 root root 1893 Aug 6 03:40 dpkg.diversions.0
-rw-r--r-- 1 root root 172 Aug 6 03:39 dpkg.statoverride.0
-rw-r--r-- 1 root root 1531155 Aug 6 03:40 dpkg.status.0
drwx----- 2 root root 4096 Sep 23 08:33 system_archive
root@debian:/home/fatima# ls -la /var/backups/system_archive
total 17140
drwx----- 2 root root 4096 Sep 23 08:33 .
drwxr-xr-x 3 root root 4096 Sep 23 06:54 ..
-rw----- 1 root root 8765851 Sep 23 08:30 backup_20260923_083019.tar.gz
-rw----- 1 root root 8770599 Sep 23 08:33 backup_20260923_083314.tar.gz
root@debian:/home/fatima#

```

Figure 1: Terminal output confirming creation and permissions of automated backup archives in /var/backups/system_archive

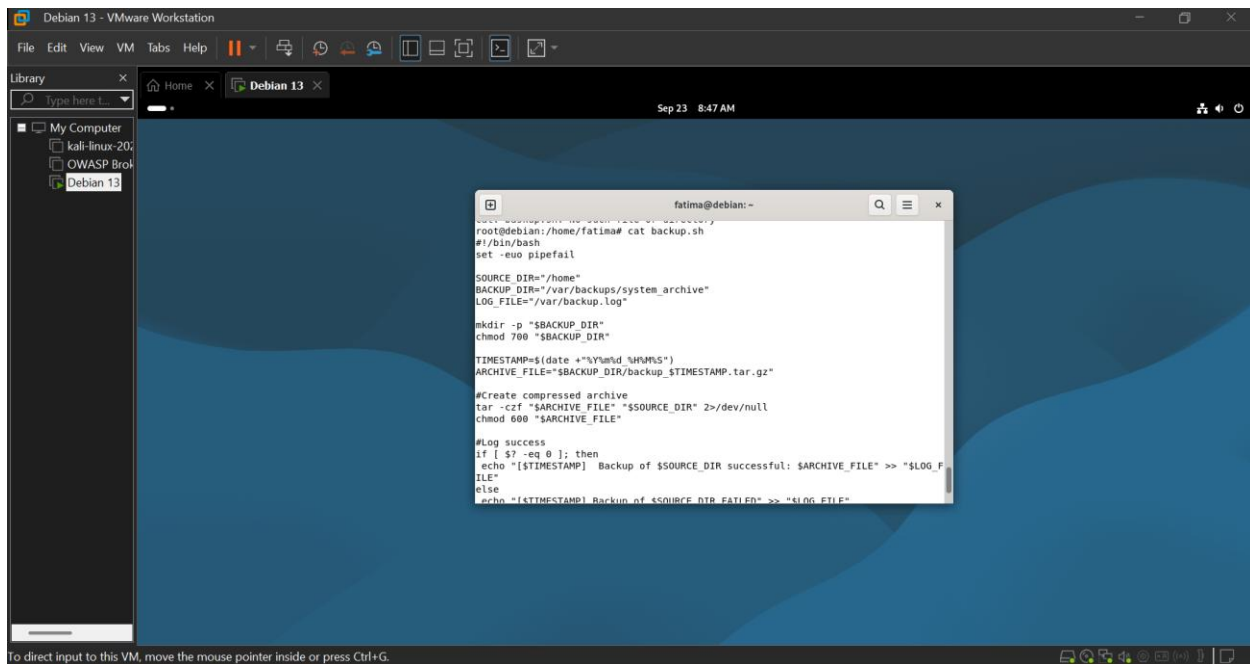


Figure 2: Initial configuration and backup execution parameters in `backup.sh`, including shell environment flags, path variables, directory creation, and compression commands.

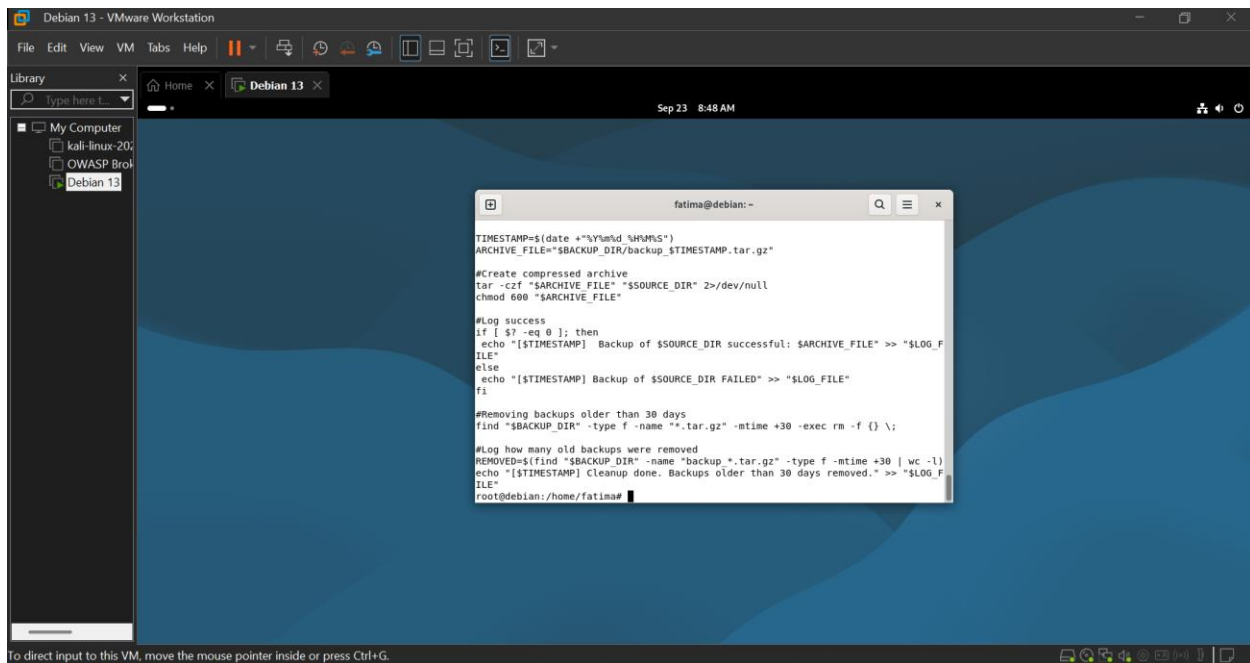


Figure 3: Execution validation and lifecycle management routines in `backup.sh`, showing exit code evaluation (`?`), audit logging, and 30-day retention cleanup using `find`.

Skill Outcomes

Through this practical exercise, I demonstrated and acquired the following technical competencies:

- **System Automation via Shell Scripting:** Writing defensive Bash scripts using environment flags, variable interpolation, and redirection operators.
- **Storage Administration:** Deploying the Linux tar utility for gzip-compressed system state archiving and inspecting archive tables of contents.
- **Linux Hardening & DAC Permissions:** Applying Discretionary Access Control (chmod 700/600) to secure storage repositories against local privilege escalation.
- **Storage Lifecycle Management:** Implementing programmatic retention schedules using find to prevent server disk saturation.
- **System Troubleshooting:** Diagnosing syntax traps in Linux shell scripting, resolving unclosed quotes, handling white-space errors in paths, and parsing execution logs.

Prerequisites

The completion of this practical application required the following technical foundations:

- Root/superuser command-line privileges within a Debian-based Linux environment.
- Working knowledge of Linux file structures (/var, /home, /root), absolute path hierarchies, and shell redirection (>, >>, 2>).
- Understanding of standard Unix file permissions (octal permissions 700, 600, and drwx representation).
- Hypervisor virtualization tools (VMware Workstation) configured for isolated server administration.